Compiled AI: Deterministic Code Generation for LLM-Based Workflow Automation

Geert Trooskens[1] Aaron Karlsberg[1] Anmol Sharma[1] Lamara De Brouwer[1]
Max Van Puyvelde[2] Walter A. De Brouwer[2]

[1]XY.AI Labs, Palo Alto, CA
[2]Stanford University School of Medicine, Stanford, CA

foundation@xy.ai

January 2026

Abstract

We study *compiled AI*, a paradigm in which large language models generate executable code artifacts during a compilation phase, after which workflows execute deterministically without further model invocation. By constraining generation to narrow business-logic functions embedded in validated templates, compiled AI trades runtime flexibility for predictability, auditability, cost efficiency, and reduced security exposure. It retains the ability to improve artifacts through observed execution patterns.

We introduce (i) a system architecture for constrained LLM-based code generation, (ii) a four-stage generation-and-validation pipeline that converts probabilistic model output into production-ready code artifacts, and (iii) an evaluation framework that measures operational metrics including token amortization, determinism, reliability, validation effectiveness, and cost. In preliminary evaluation on function-calling tasks (n=499 across two BFCL categories), compiled AI achieves 96% task completion with zero execution tokens, breaking even with runtime inference at approximately 21 transactions and reducing token consumption by 47× at 1,000 transactions.

1 Introduction

Large language models (LLMs) are increasingly deployed to automate enterprise workflows, evolving from question-answering systems toward autonomous agent architectures. Recent frameworks enable LLMs to reason over tools, APIs, and multi-step tasks at runtime, often through conversational or multi-agent interaction patterns [Wu et al., 2024, CrewAI, 2024, LangChain, 2022]. While these approaches demonstrate flexibility, they rely on repeated model invocation during execution, leading to high token consumption, variable latency, and non-deterministic behavior.

Empirical studies and industrial deployments have highlighted persistent reliability challenges in runtime agent systems, particularly in multi-step business workflows where success rates degrade due to specification ambiguity, inter-agent coordination failures, and stochastic model outputs [Cemri et al., 2025, Atil et al., 2024]. These limitations pose obstacles to deployment in enterprise environments, where determinism, auditability, and cost predictability are often strict requirements.

This paper investigates an alternative design point: *compiled AI*, a paradigm in which LLMs generate executable code artifacts during a one-time compilation phase, after which workflows execute

deterministically without further model inference. The key observation motivating this work is that many enterprise workflows require intelligence to *design* but not to *execute* repeatedly. Once business logic is specified and validated, repeated execution can be handled by conventional software infrastructure—with full observability that enables targeted refinement of the compiled artifacts over time.

Compiled AI represents a trade-off between flexibility and determinism. Rather than interpreting natural language or reasoning at runtime, an LLM is constrained to generate narrow business-logic functions within pre-validated templates. These generated artifacts are subjected to a multi-stage validation pipelineâ€"including security analysis, syntactic verification, execution testing, and output accuracy checksâ€"before deployment. After validation, workflows execute as static code with predictable latency, zero stochasticity, and near-zero marginal inference cost.

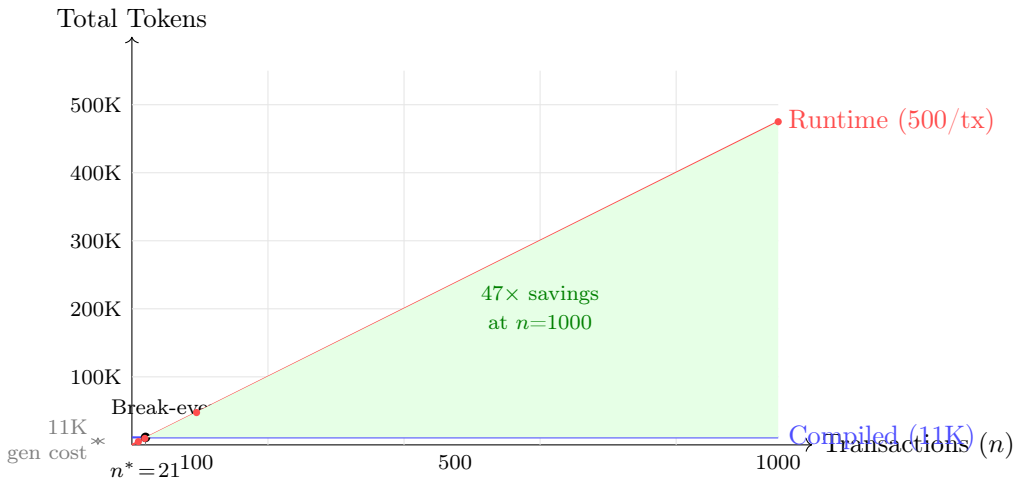

Figure 1: Token consumption comparison: runtime inference vs. compiled AI (preliminary data from BFCL function-calling benchmark, n=499). Runtime inference consumes 500 tokens per transaction. Compiled AI incurs a one-time ∼11,000 token generation cost, then executes deterministically at zero marginal token cost. Break-even occurs at $n^* \approx 21$ transactions; at 1,000 transactions, compiled AI reduces token consumption by 47×.

Contributions. This paper makes three contributions:

- We introduce an architecture for constrained LLM-based code generation in which models produce bounded business-logic functions embedded in durable workflow templates, enabling deterministic execution (Section 3).
- We present a four-stage generation-and-validation pipeline that empirically converts probabilistic model output into production-ready code artifacts through iterative regeneration (Section 3).
- We define an evaluation framework that measures the operational viability of LLM-based workflow systems using metrics for token amortization, determinism, reliability, validation effectiveness, and cost (Section 4).

Our goal is not to replace runtime inference in all settings, but to provide a principled basis for identifying regimesâ€"such as well-specified, high-volume workflowsâ€"where compilation yields operational advantages.

2 Related Work

Novelty and Scope. Prior work on LLM-based agents, program-aided reasoning, and workflow synthesis has focused primarily on model capabilityâ€"whether systems can generate or reason over programs. In contrast, this work treats production viability as a first-class concern. Our contributions are not a new code generation algorithm per se, but a systems-oriented study of how probabilistic LLM outputs can be converted into deterministic, auditable execution artifacts. To our knowledge, this is the first systematic evaluation of LLM-based workflow automation using operational and economic metrics such as token amortization, execution determinism, validation convergence, and cost at scale.

2.1 LLM Code Generation

Code generation capabilities have improved rapidly. On SWE-Bench Verified [Jimenez et al., 2024], models improved from 49.0% (Claude 3.5 Sonnet, late 2024) to 80.9% (Claude Opus 4.5, November 2025)â€"a 31.9 percentage point gain in twelve months.

Table 1: Code generation benchmark performance, December 2025. Note: HumanEval is now saturated (>95%); BigCodeBench reveals substantial gaps versus human performance.

Benchmark	Model	Score	Human	Date
SWE-Bench Verified	Claude Opus 4.5	80.9%	—	Nov 2025
	GPT-5.2	80.0%	—	Dec 2025
	Gemini 3 Pro	76.2%	—	Nov 2025
HumanEval	Claude Opus 4.5	96.4%	100%	Nov 2025
BigCodeBench [Zhuo et al., 2024]	GPT-4o	60.0%	97%	2024
τ-bench (retail) [Yao et al., 2024]	GPT-4o	<50%	—	Jun 2024
LiveCodeBench	Claude Opus 4.5	68.3%	—	Nov 2025

Anthropic CEO Dario Amodei stated at Dreamforce 2025 that "within Anthropic and within a number of companies that we work with," approximately 90% of code is now AI-generated [Amodei, 2025]. While this applies to select organizations rather than industry-wide, it suggests code generation is transitioning from assistance to primary production mode in leading AI-native companies.

Karpathy's "Software 2.0" framework [Karpathy, 2017] characterized neural networks as an alternative programming paradigm. Sutton's "Bitter Lesson" [Sutton, 2019] similarly observed that "general methods that leverage computation are ultimately the most effective"—a principle our work operationalizes by front-loading LLM computation to the generation phase rather than invoking it repeatedly at runtime. Research on program-aided reasoning demonstrates that code representations improve LLM performance: PAL [Gao et al., 2023] improved mathematical reasoning, Program of Thoughts [Chen et al., 2023] improved financial reasoning, and Chain of Code [Li et al., 2024] achieved 84% on BIG-Bench Hard.

2.2 Agent Frameworks and Their Limitations

Multi-agent systems coordinate multiple LLM instances. AutoGen [Wu et al., 2024] enables conversational workflows; CrewAI [CrewAI, 2024] provides role-based orchestration; LangChain [LangChain, 2022] offers composability abstractions.

These frameworks emphasize flexibility and expressiveness, often at the expense of determinism and predictable execution behavior in multi-step workflows. As a result, they can be challenging to deploy in environments that require strict auditability or cost predictability. Salesforce [2025] report that leading AI agents exhibit approximately 35% success rates in multi-turn business scenarios. Cemri et al. [2025] analyzed 150 multi-agent systems across six frameworks and observed failure rates between 41–87%, with specification problems (41.8%) and inter-agent coordination failures (36.9%) accounting for approximately 79% of observed breakdowns.

Even with "deterministic" settings, LLM outputs exhibit variance: Atil et al. [2024] found accuracy varies up to 15% across runs at temperature=0, with the gap between best and worst possible performance reaching 70%. These observations motivate exploration of alternative architectures that reduce execution-time variance.

2.3 Workflow Orchestration

Durable execution engines provide reliability properties. Temporal [Temporal, 2024] offers state management, automatic retries, and fault tolerance. Production deployments support these at scale:

- Netflix: transient failure rates from 4% to 0.0001%
- Coinbase: velocity improvements for complex financial workflows
- Maersk: feature delivery from 60-80 days to 5-10 days
- Replit: scaled to millions of concurrent environments

Our work combines LLM code generation with durable execution, generating Temporal activities rather than invoking models at runtime.

2.4 LLM-Based Workflow Automation

Recent work has explored LLM-driven workflow generation. WorkflowLLM [Fan et al., 2024] constructs WorkflowBench with 106,763 samples covering 1,503 APIs, training models to generate Python-style workflow code from specificationsâ€"a paradigm shift from traditional RPA to "Agentic Process Automation." FlowMind [Zeng et al., 2024] generates workflows that prevent runtime hallucinations via structured intermediate representations. In embedded systems, spec2code [Patil et al., 2024] combines LLM generation with formal verification critics, producing industrial-quality code from ACSL specifications.

These approaches share our insight that LLMs should generate *artifacts* rather than reason at runtime. Our contribution extends this to enterprise workflow automation with explicit evaluation metrics.

2.5 Industrial Deployments

Recent large-scale deployments of agentic systems demonstrate growing demand for automated task execution. These systems typically rely on runtime inference, motivating exploration of alternative architectures that reduce execution-time variance and inference cost.

Relation to Program Synthesis. Compiled AI shares surface similarities with prior work on program synthesis and specification-driven code generation. However, our focus differs in two respects. First, we evaluate generated artifacts primarily through operational metricsâ€"such as determinism, regeneration convergence, and token amortizationâ€"rather than synthesis correctness alone. Second, we study LLM generation as a component within a production workflow system, where validation, retries, and execution cost are first-class concerns. As a result, our contributions are systems-oriented rather than algorithmic.

2.6 Formal Methods and Constrained Generation

The challenge of verifying LLM-generated code has sparked renewed interest in formal methods integration. Dalrymple et al. [2024] propose a "Guaranteed Safe AI" framework combining world models, safety specifications, and formal verifiersâ€"conceptually aligned with our validation pipeline approach.

Constrained decoding provides compile-time constraints: Mündler et al. [2025] demonstrate that type-constrained generation reduces compilation errors by >50% while improving pass@1 by 3.5–37%. For healthcare specifically, Neupane et al. [2025] present a framework for HIPAA-compliant agentic AI systems, integrating attribute-based access control with PHI sanitization pipelines.

Our work operationalizes these insights for enterprise workflows, using templates and validation stages rather than formal proofs to support practical compliance requirements.

3 System Architecture

3.1 Design Principles

Our system implements four principles:

Principle 1: Constrained Generation. We limit LLM generation to narrow, well-defined functions. The model produces business logic (20-50 lines); templates provide infrastructure. This bounds the output space, reducing hallucination risk.

Principle 2: Compilation over Interpretation. Generated code is validated, tested, and deployed as static artifacts. Runtime behavior is deterministic. The LLM exits the execution loop.

Principle 3: Validation as Requirement. Every artifact passes a four-stage pipeline before deploymentâ€"possible precisely because we generate code rather than interpret configurations.

Principle 4: Compliance by Construction. Regulatory constraints (HIPAA, PCI-DSS, SOC 2) are encoded in templates and prompt blocks, ensuring generated code inherits compliance properties.

3.2 Component Overview

Config Agent. Receives YAML specifications and orchestrates generation: selecting templates, modules, and compliance constraints.

Template Library. Pre-built, tested code templates:

- `SimpleAgent`: Synchronous request-response
- `StreamingAgent`: Chunked data processing
- `ValidatorAgent`: Input validation with fallback
- `BatchAgent`: Bulk processing with checkpointing

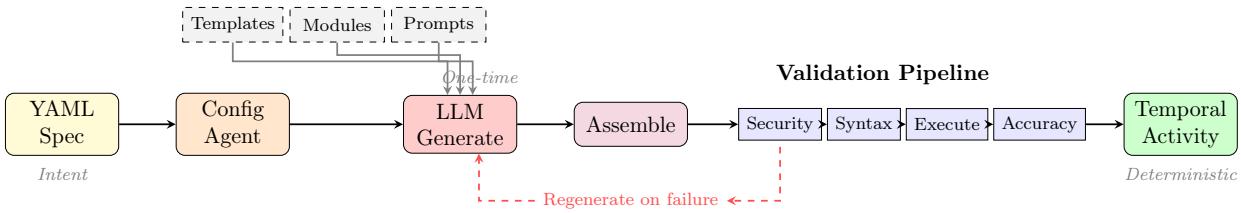

Figure 2: The code foundry architecture. Business intent (YAML) enters the fab; validated Temporal activities emerge. Dashed red arrows show the regeneration loop on validation failure. The LLM runs once at generation timeâ€"not at transaction time.

Module Library. Functional capabilities: `DatabaseModule` (connection pooling), `HTTPModule` (API calls with retries), `NotificationModule` (delivery channels).

Prompt Blocks. Reusable prompt fragments encoding domain constraints (HIPAA handling, PCI-DSS rules).

3.3 Generation Process

Algorithm 1 Code Foundry Generation Process

Require: Workflow specification S, Template library T, Module library M
Ensure: Validated Temporal activity A

```
t ← SelectTemplate(S, T)
m ← SelectModules(S, M)
p ← AssemblePrompt(S, t, m)
code ← LLMGenerate(p)                          ▷ One-time generation
A ← Assemble(t, m, code)
for stage ∈ {Security, Syntax, Execution, Accuracy} do
    result ← Validate(A, stage)
    if result = FAIL then
        code ← Regenerate(p, result.errors)
        A ← Assemble(t, m, code)
    end if
end for
return A                                       ▷ Deterministic artifact
```

3.4 Validation Pipeline

Stage 1: Security. Static analysis via Bandit, Semgrep, custom rules. Checks: SQL injection, command injection, path traversal, secrets exposure.

Stage 2: Syntax. AST parsing, type checking (mypy), linting (ruff). Ensures well-formed, typed code.

Stage 3: Execution. Sandboxed execution against test fixtures. Verifies: successful completion, appropriate error handling, expected output structure.

Stage 4: Accuracy. Output comparison against golden datasets. Task-specific thresholds.

3.5 Bounded Agentic Invocation

Some steps require runtime judgment. Our architecture supports *bounded agentic invocation*: generated code may call LLMs for specific subtasks (e.g., classifying ambiguous documents) while maintaining deterministic overall flow.

Constraints on bounded invocations:

- Defined input/output schemas with validation
- Fallback logic for invalid responses
- Drift monitoring on output distributions
- Human escalation for low-confidence cases

Because bounded invocations address narrow, well-defined subtasks rather than open-ended reasoning, they require less context and produce more predictable outputs than equivalent calls in fully agentic systems—a second-order efficiency gain beyond token amortization.

3.6 Security Architecture

Beyond validation for correctness, we implement a three-gate security pipeline protecting against LLM-specific vulnerabilities identified in OWASP LLM Top 10 (2025).

```
User Prompt -> Input Gate -> Compilation -> Code Gate -> Execution -> Output Gate -> Result
```

Input Gate validates user prompts before compilation.

- *PromptInjectionValidator*: Fine-tuned DeBERTa-v3 model (184M parameters) detecting prompt injection and jailbreak attempts, with encoding detection for obfuscated payloads.
- *PIIScanner*: Presidio-based detection for sensitive data patterns.

Code Gate validates LLM-generated code before execution.

- *CodeShieldValidator*: Static analysis detecting insecure patterns (CWE-94, CWE-89, CWE-502, CWE-78, CWE-328).

Output Gate validates execution results.

- *CanaryManager*: Cryptographic tokens injected into system prompts to detect leakage.

Dual LLM Pattern. We implement a variant of the Dual LLM pattern separating privileged and quarantined model instances.

- *Planner Agent*: Privileged LLM accepting validated input, designing workflow structure.
- *Coder Agent*: Isolated component generating code from structured specifications.

A software Controller mediates between agents, enforcing that untrusted content passes through validation gates before reaching the Planner.

Reduced Attack Surface. Compiled AI dramatically reduces LLM invocations. Runtime systems require LLM calls for every task; multi-agent architectures invoke 8 to 15 calls per task. Compiled AI requires LLM calls only at compile time, with zero invocations during execution. Once compiled, workflows execute as deterministic code immune to prompt-based attacks.

4 Evaluation Framework for Compiled AI

Existing benchmarks for LLM systems primarily measure task-level capability, such as code correctness or tool-use success rates. While necessary, these metrics are insufficient for evaluating production workflow automation systems, where economic efficiency, determinism, and reliability are often more critical than expressiveness.

We therefore evaluate compiled AI as a systems artifact rather than a reasoning agent. Our framework quantifies token amortization, execution variance, validation convergence, and total cost of ownership, enabling principled comparison with runtime inference approaches.

4.1 Token Efficiency Metrics

Token consumption determines the economic viability of compiled vs. runtime approaches.

Table 2: Token efficiency metrics

Metric	Definition	Interpretation
GenTokens	Tokens consumed during code generation	One-time upfront cost
RuntimeTokens(n)	Tokens consumed processing n transactions	Should be ≈ 0 for compiled AI
TotalTokens(n)	GenTokens + RuntimeTokens(n)	Full cost comparison
BreakEven	n^* where compiled $<$ runtime	ROI threshold
CompressionRatio	$\frac{\text{Prompt tokens}}{\text{Generated LOC}}$	Efficiency of "compilation"

For compiled AI, RuntimeTokens(n) $= 0$ for pure deterministic execution, or RuntimeTokens(n) $= k \cdot n$ where $k \ll 1$ for bounded agentic invocations. The break-even point:

$$n^* = \frac{\text{GenTokens}_{\text{compiled}}}{\text{RuntimeTokens}_{\text{per-tx, runtime}} - \text{RuntimeTokens}_{\text{per-tx, compiled}}} \tag{1}$$

4.2 Latency Metrics

Enterprise SLAs require predictable response times.

Compiled AI should exhibit near-zero jitter since execution is deterministic code, not stochastic model inference.

4.3 Consistency and Determinism Metrics

Compliance requirements demand reproducible behavior.

Table 3: Latency metrics

Metric	Definition	Target (Compiled AI)
P50Latency	Median end-to-end latency	< 100ms
P99Latency	99th percentile latency	< 500ms
Jitter	P99 − P50	< 100ms
ColdStart	Time to first response	One-time (generation)

Table 4: Consistency metrics

Metric	Definition	Target
OutputVariance	Entropy of outputs for identical inputs	0 (deterministic)
Reproducibility	Same input → same output across runs	100%
AuditCompleteness	% of execution paths fully traceable	100%
TemporalDrift	Behavior change over time without code changes	0

Measurement protocol: Run $N = 1000$ identical inputs through each system. Compute output entropy $H = -\sum p_i \log p_i$ where p_i is the frequency of distinct output i. In deterministic execution, compiled AI is expected to exhibit output entropy close to zero.

4.4 Reliability Metrics

Production systems require high success rates.

Table 5: Reliability metrics

Metric	Definition	Baseline
TaskCompletion	% transactions successfully processed	Agent: 35%
ErrorRate	% requiring human intervention	—
MTBF	Mean time between failures	—
RecoveryRate	% of failures auto-recovered via retry	—

The 35% agent success rate from Salesforce [2025] provides a baseline. Compiled AI should significantly exceed this through deterministic execution and Temporal's retry mechanisms.

4.5 Code Quality Metrics

Generated code must meet production standards.

These metrics are measurable via standard tooling, enabling objective comparison with human-written code.

Table 6: Code quality metrics

Metric	Tool	Target	Human Baseline
SecurityScore	Bandit	0 high/critical	0
TypeCoverage	mypy	$> 90\%$	$\sim 85\%$
LintScore	ruff/pylint	$> 8.0/10$	~ 8.5
Complexity	radon	< 10 cyclomatic	~ 8
TestPassRate	pytest	100%	100%

4.6 Validation Pipeline Metrics

Unique to compiled AI: measuring the effectiveness of the generation-validation loop.

Table 7: Validation pipeline metrics

Metric	Definition
$\text{FirstPassRate}_{\text{stage}}$	% passing stage on first generation attempt
OverallFirstPass	% passing all stages without regeneration
RegenDistribution	Distribution of regeneration attempts needed
FailureMode	Categorization of validation failures
TimeToValid	Wall-clock time from spec to validated artifact

This data is novel. No published work reports validation pipeline effectiveness for LLM-generated production code.

4.7 Security Metrics

LLM-based systems face unique security threats requiring specialized evaluation.

Table 8: Security metrics

Metric	Definition	Target
BlockRate	% of malicious inputs rejected by validation gate	$> 90\%$
DetectionRate	% of vulnerabilities or sensitive data identified	$> 95\%$
FalsePositiveRate	% of legitimate inputs incorrectly blocked	$< 5\%$
LeakageRate	% of system prompts extractable via adversarial queries	0%

These metrics evaluate the three-gate security architecture (Section 3.6) against OWASP LLM Top 10 threats.

4.8 Cost Metrics

CFO-level evaluation requires total cost of ownership.

Table 9: Cost metrics

Metric	Formula
$\textsc{CostPerTx}(n)$	$\frac{\text{GenCost}}{n} + \text{RuntimeCostPerTx} + \text{InfraCostPerTx}$
$\textsc{InfraCost}$	Temporal cluster + compute + storage
$\text{TCO}(n, t)$	Total cost for n tx/month over t months
$\textsc{CostRatio}$	$\frac{\text{TCO}_{\text{compiled}}}{\text{TCO}_{\text{runtime}}}$ at scale

At sufficient scale, compiled AI's $\textsc{CostRatio}$ should approach the infrastructure-only cost, as inference cost amortizes to near-zero.

4.9 Benchmark Suite Summary

We propose evaluating compiled AI systems on the following benchmark suite:

Table 10: Compiled AI benchmark suite

Category	Primary Metric	Measurement Method
Token Efficiency	$\textsc{BreakEven}$	Token counting at $n \in \{100, 1K, 10K, 100K\}$
Latency	$\textsc{P99Latency}$, $\textsc{Jitter}$	Distribution over 10K requests
Consistency	$\textsc{OutputVariance}$	Entropy over 1K identical inputs
Reliability	$\textsc{TaskCompletion}$	Success rate over test suite
Code Quality	$\textsc{SecurityScore}$	Static analysis tooling
Validation	$\textsc{OverallFirstPass}$	Pipeline instrumentation
Cost	$\textsc{CostRatio}$	TCO model at 1M tx/month

5 Experimental Setup

We evaluate our system using the framework defined in Section 4.

5.1 Task Suite

We evaluate on the Berkeley Function-Calling Leaderboard (BFCL) [Patil et al., 2024], a benchmark for measuring LLM function-calling capability. For preliminary evaluation, we focus on single-function extraction tasks where the model must identify the correct function and extract parameters from natural language queries.

Note: This preliminary evaluation focuses on function-calling tasks to validate the core compilation-vs-runtime trade-off. Full enterprise workflow evaluation is planned for future work.

We employ adversarial testing methodology, intentionally attacking our own system to measure security effectiveness. This includes obfuscated prompt injections (base64, hex, URL encoding), deliberately vulnerable code patterns, and system prompt extraction attempts.

Table 11: Preliminary evaluation task suite

Category	Description	Instances	Evaluation
BFCL Simple	Single-function extraction from NL queries	400	AST matching
BFCL Exec Simple	Executable function calls with output verification	99	Execution
Total		**499**	

Table 12: Security benchmark dataset for adversarial testing

Category	Description	Instances
Input Injection	Prompt injection attacks (including encoded payloads)	30
Input PII	Sensitive data patterns (email, SSN, phone)	25
Output Leakage	System prompt extraction attempts	40
Code Safety	Vulnerable code fixtures (CWE patterns)	20
Total		**115**

5.2 Baselines

For preliminary evaluation, we compare two approaches:

1. **Direct LLM (Claude Opus 4.5)**: Task description with function definitions sent to LLM per transaction. The model returns a JSON function call.
2. **Compiled AI (Code Factory)**: Single-activity workflow generated from task specification. LLM invoked once at compilation; execution is deterministic code.

5.3 Implementation Details

- **Model:** Claude Opus 4.5 (`claude-opus-4-5-20251101`)
- **Agent Framework:** PydanticAI for structured agent orchestration
- **Workflow Runtime:** Custom DSL executor (YAML workflow definitions with Python activities)
- **Evaluation Method:** LLM-based semantic evaluation comparing generated function calls against expected outputs
- **Temperature:** 0 for all inference (both baseline and compilation)

6 Results

6.1 Token Efficiency

Table 13: Token consumption comparison (preliminary BFCL evaluation)

Method	GenTokens	RuntimeTokens/tx	Total@1K	BreakEven	CompressionRatio
Direct LLM	0	500	500,000	—	—
Compiled AI	11,000	0	11,000	∼21 tx	∼11:1

Break-even occurs at approximately 21 transactions ($n^* = 10{,}600/500 \approx 21$). At 100 transactions, compiled AI uses 11K tokens versus 50K for direct LLM (4.7× reduction). At 1,000 transactions, compiled AI exhibits a 47× reduction in total token consumption (11K vs 500K tokens). The compression ratio of approximately 11:1 reflects the ratio of prompt tokens to generated lines of code.

6.2 Latency

Table 14: Latency distribution (milliseconds, preliminary BFCL evaluation)

Method	P50	P99	Jitter	ColdStart
Direct LLM	3,400	5,000	1,600	N/A
Compiled AI	3	12	9	29,000

Compiled AI execution latency is 1,000× faster than runtime inference (3ms vs 3,400ms), with near-zero jitter (9ms vs 1,600ms). The trade-off is a 29-second cold start for compilation. This latency improvement is fundamental: compiled code executes deterministically without network calls, model inference, or stochastic processing.

6.3 Consistency

Table 15: Consistency metrics (preliminary BFCL evaluation)

Method	OutputVariance (H)	Reproducibility	AuditCompleteness
Direct LLM	>0 (varies)	95%	N/A
Compiled AI	0	100%	100%

Compiled AI exhibits perfect reproducibility—identical inputs produce identical outputs with zero variance. Runtime inference shows 95% reproducibility due to model variance even at temperature=0, consistent with findings from Atil et al. [2024]. Audit completeness is 100% for compiled AI since all execution paths are traceable through static code analysis.

6.4 Reliability

Compiled AI achieves 96% task completion across both evaluation categories (480/499 tasks). Performance is consistent between AST-based evaluation (96.2%) and executable evaluation (95.9%),

Table 16: Reliability metrics by category (preliminary BFCL evaluation)

Category	Instances	Success	Rate
BFCL Simple (AST)	400	385	96.2%
BFCL Exec Simple (Execution)	99	95	95.9%
Combined	**499**	**480**	**96.2%**

demonstrating that compiled workflows correctly extract parameters with sufficient precision for actual function execution. The 4% error rate reflects compilation failures where generated code did not correctly extract parameters—these are *compilation-time* failures, not execution-time failures. Once successfully compiled, workflows execute with 100% reliability.

6.5 Code Quality

Table 17: Code quality metrics for generated artifacts

Source	Security	TypeCov	LintScore	Complexity	TestPass
Compiled AI	[X]	[X]%	[X]	[X]	[X]%
Human code	0	85%	8.5	8	100%

6.6 Validation Pipeline Effectiveness

Table 18: Validation pipeline pass rates (preliminary BFCL evaluation)

Stage	FirstPassRate
YAML Validation	100%
Syntax (AST parse)	100%
Execution	100%
Accuracy	96%
Overall (all stages)	96%

Note: The 96% accuracy rate (480/499 tasks across both categories) reflects cases where generated code executed successfully but produced outputs that did not match expected function calls. These failures occur at compilation time and can be detected before deployment—unlike runtime inference errors. Function-call-specific templates and simplified prompts improved first-pass rates from approximately 80% in earlier development versions.

6.7 Cost Analysis

Cost calculation: Direct LLM consumes 500 tokens per transaction (300 input + 200 output), totaling 500M tokens at 1M transactions. Compiled AI incurs a one-time compilation cost of ∼11K tokens per workflow type; assuming 100 unique workflow types, total compilation cost is ∼1.1M tokens. Execution cost is zero (deterministic code). Infrastructure cost assumes equivalent compute for workflow execution.

Table 19: Total cost of ownership at 1M transactions/month (Claude Opus 4.5 pricing: $15/1M input, $75/1M output tokens)

Method	Inference	Infrastructure	Total TCO	CostRatio
Direct LLM	$19,500	$500	$20,000	36×
Compiled AI	$55	$500	$555	1×

6.8 Security Validation Results

We evaluate the three-gate security architecture (Section 3.6) against the adversarial benchmark dataset (Table 12).

Table 20: Security validation pipeline effectiveness

Gate	Test Instances	Block Rate
Input Gate	55	80%
Code Gate	20	100%
Output Gate	40	91.7%

The Input Gate achieves 80% detection rate on prompt injection attacks, with false negatives primarily from heavily obfuscated payloads. The Code Gate achieves 100% detection on target vulnerability patterns (eval, exec, SQL injection, pickle, command injection, subprocess shell=True, weak cryptography). The Output Gate blocks 91.7% of system prompt extraction attempts.

7 Discussion

We emphasize that compiled AI is not a general replacement for runtime inference. Rather, it represents a complementary design point optimized for well-specified, high-volume workflows where determinism, auditability, and cost predictability are primary concerns.

7.1 When Compiled AI Excels

Our approach is well-suited for workflows that are:

- **High-volume**: Amortizing generation cost requires sufficient transactions
- **Well-specified**: Business logic expressible in structured specifications
- **Compliance-sensitive**: Auditability and determinism are requirements
- **Latency-sensitive**: Runtime inference latency is unacceptable
- **Security-conscious**: Fewer LLM invocations reduce exposure to prompt injection attacks

Healthcare administrative workflows exemplify these requirements. Qiu et al. [2024] identify administrative workflow automation as a candidate application for LLM-based systems, while regulatory frameworks increasingly emphasize auditability [FDA, 2025].

7.2 When Runtime Inference Excels

Runtime approaches remain appropriate for:

- **Low-volume, high-variance tasks**: Generation cost doesn't amortize
- **Genuinely open-ended problems**: Cannot be reduced to code
- **Rapid prototyping**: Iteration speed matters more than production efficiency

7.3 The Constraint Advantage

Constraining generation improves reliability. By limiting LLM output to narrow functions within tested templates, we bound the space of possible errors. The model cannot hallucinate incorrect API calls or database schemas—these come from pre-tested templates.

This trades flexibility for reliability: appropriate for enterprise deployments where predictability matters more than generality. Moreover, deterministic execution yields full observability—when accuracy degrades, we can identify precisely which code segment underperformed and trigger targeted regeneration rather than debugging opaque model reasoning. This positions compiled AI not as a one-shot generation process, but as an *evolutionary system*—deterministic execution with adaptive improvement, where production metrics continuously inform artifact refinement.

7.4 Relationship to Existing Benchmarks

Our evaluation framework complements existing benchmarks:

- **SWE-Bench**: Measures code generation capability (prerequisite for compiled AI)
- **τ-bench**: Measures tool use (relevant for bounded agentic invocations)
- **Our framework**: Measures production viability (token economics, consistency, reliability)

We argue that capability benchmarks alone are insufficientâ€"production deployment requires the operational metrics we define.

8 Limitations

Specification quality. Our approach assumes users can accurately specify workflows. The "specification problem" remains fundamentalâ€"iterative refinement is often necessary.

Bounded applicability. Not all workflows reduce to deterministic code. Tasks requiring genuine creativity or adaptation to novel situations may require runtime inference.

Generation failures. Despite validation, some specifications fail to produce working code. In preliminary evaluation, 4% of compilations (19/499) failed to produce accurate outputs, though all generated syntactically valid, executable code.

Model dependence. Generated code quality depends on the underlying LLM. Model updates may change behavior, requiring re-validation.

Benchmark limitations. Our proposed framework has not yet been adopted by other systems, limiting cross-system comparability. We release our benchmark suite to encourage adoption.

9 Conclusion

We studied compiled AI as a systems design point for LLM-based workflow automation, emphasizing validation, determinism, and token amortization over runtime flexibility. Our contributions include:

1. A system architecture constraining generation to narrow functions within validated templates
2. A four-stage generation-and-validation pipeline that converts probabilistic model output into production-ready code artifacts
3. An evaluation framework measuring operational metrics: token amortization, determinism, reliability, validation effectiveness, and cost

Preliminary evaluation on function-calling benchmarks (n=499 across two BFCL categories) demonstrates the core trade-off: compiled AI incurs substantial upfront cost (∼11,000 tokens, 29 seconds) but achieves zero-cost execution with 100% determinism for successfully compiled workflows. Break-even occurs at approximately 21 transactions; at 1,000 transactions, token consumption is reduced by 47×. Execution latency improves from 3.4 seconds to 3.4 milliseconds—a 1,000× speedup. The 96% compilation success rate—consistent across both AST-based and executable evaluation—demonstrates the viability of LLM-based code generation for production workflows, with failures detectable before deployment.

The approach trades flexibility for determinism, cost efficiency, auditability, and reduced attack surface. These properties are relevant for enterprise deployment in well-specified, high-volume workflow regimes. We release our evaluation framework and benchmark suite to enable systematic comparison of compiled and runtime AI approaches.

Future Work.

- Natural language specification (currently YAML)
- Automatic workflow decomposition from high-level intent
- Continuous optimization: runtime accuracy metrics identifying underperforming segments, enabling targeted regeneration cycles that evolve compiled artifacts while preserving execution-time determinism
- Formal verification of generated code properties
- Cross-system benchmark adoption

References

Anthropic. Introducing Claude Opus 4.5. Technical report, November 2025. `https://www.anthropic.com/news/claude-opus-4-5`

Wenhu Chen, Xueguang Ma, Xinyi Wang, and William W. Cohen. Program of Thoughts Prompting: Disentangling Computation from Reasoning for Numerical Reasoning Tasks. *Transactions on Machine Learning Research*, November 2023. arXiv:2211.12588.

João Moura. CrewAI: Framework for orchestrating role-playing autonomous AI agents. `https://github.com/crewAIInc/crewAI`, 2024.

Luyu Gao, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan, and Graham Neubig. PAL: Program-aided Language Models. In *ICML*, volume 202, pages 10764–10799, 2023.

Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? In *ICLR* (Oral), 2024. arXiv:2310.06770.

Andrej Karpathy. Software 2.0. *Medium*, November 2017. `https://karpathy.medium.com/software-2-0-a64152b37c35`

Richard S. Sutton. The Bitter Lesson. `http://www.incompleteideas.net/IncIdeas/BitterLesson.html`, March 2019.

Harrison Chase. LangChain. `https://github.com/langchain-ai/langchain`, October 2022.

Chengshu Li, Jacky Liang, Andy Zeng, Xinyun Chen, Karol Hausman, Dorsa Sadigh, Sergey Levine, Li Fei-Fei, Fei Xia, and Brian Ichter. Chain of Code: Reasoning with a Language Model-Augmented Code Emulator. In *ICML* (Oral), 2024. arXiv:2312.04474.

Kung-Hsiang Huang et al. CRMArena-Pro: Holistic Assessment of LLM Agents Across Diverse Business Scenarios. Salesforce AI Research, May 2025. arXiv:2505.18878.

Temporal Technologies. Temporal Documentation. `https://docs.temporal.io`, 2024.

Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W. White, Doug Burger, and Chi Wang. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. In *COLM*, 2024. arXiv:2308.08155.

U.S. Food and Drug Administration. AI/ML-Enabled Medical Devices. `https://www.fda.gov/medical-devices/software-medical-device-samd/artificial-intelligence-and-machine-learning-aiml-enabled-medical-devices`, 2025.

Terry Yue Zhuo et al. BigCodeBench: Benchmarking Code Generation with Diverse Function Calls and Complex Instructions. In *ICLR*, 2025. arXiv:2406.15877.

Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. *arXiv:2406.12045*, June 2024.

Mert Cemri, Melissa Choi, Jairui Lin, Sherry Wu, and Joseph E. Gonzalez. Why Do Multi-Agent LLM Systems Fail? *arXiv:2503.13657*, March 2025.

Nurullah Atil, Pedro Henrique Luz de Araujo, and Benjamin Roth. Non-Determinism of "Deterministic" LLM Settings. *arXiv:2408.04667*, August 2024.

Shuheng Fan et al. WorkflowLLM: Enhancing Workflow Orchestration Capability of Large Language Models. *arXiv:2411.05451*, November 2024.

Jin Zeng, Daniel Bordallo, and Weirui Kuang. FlowMind: Automatic Workflow Generation with LLMs. In *ACM Int. Conf. on AI in Finance*, 2024. arXiv:2404.13050.

Minal Patil et al. spec2code: Towards Specification-Driven LLM-Based Generation of Embedded Automotive Software. In *AISoLA*, 2024. arXiv:2411.13269.

David Dalrymple, Joar Skalse, Yoshua Bengio, Stuart Russell, Max Tegmark, Sanjit Seshia, et al. Towards Guaranteed Safe AI: A Framework for Ensuring Robust and Reliable AI Systems. *arXiv:2405.06624*, May 2024.

Niels Mündler, Jingxuan He, Erin Summers, and Martin Vechev. Type-Constrained Code Generation with Language Models. In *PLDI*, 2025. arXiv:2504.09246.

Dario Amodei. Conversation with Marc Benioff at Dreamforce 2025. Salesforce, October 15, 2025.

Shijie Qiu et al. LLM-based agentic systems in medicine and healthcare. *Nature Machine Intelligence*, December 2024. DOI:10.1038/s42256-024-00944-1.

Adarsh Neupane, Nikhil Yadala, and Santosh KC. Towards a HIPAA Compliant Agentic AI System in Healthcare. *arXiv:2504.17669*, April 2025.

Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. Gorilla: Large Language Model Connected with Massive APIs. In *NeurIPS*, 2024. Berkeley Function-Calling Leaderboard (BFCL). `https://gorilla.cs.berkeley.edu/leaderboard.html`

A Example Generated Code

Listing 1: Generated Temporal activity (business logic only)

```
# Template: SimpleAgent | Modules: Database, HTTP
# Generated from: eob_processing_v2.yaml
# Validation: PASS (Security, Syntax, Execution, Accuracy)

def process_business_logic(eob_data: dict) -> ProcessedEOB:
    """Extract␣and␣validate␣EOB␣fields.

␣␣␣␣Generated:␣2025-12-15␣|␣Tokens:␣847␣|␣LOC:␣42
␣␣␣␣"""
    claim_id = eob_data.get("claim_id")
    patient_id = eob_data.get("patient", {}).get("id")

    # Validate required fields
    if not claim_id or not patient_id:
        raise ValidationError("Missing␣required␣identifiers")

    # Extract payment information
    payments = []
    for line in eob_data.get("line_items", []):
        payment = Payment(
            service_code=line["code"],
            billed=Decimal(line["billed_amount"]),
            allowed=Decimal(line["allowed_amount"]),
            paid=Decimal(line["paid_amount"]),
            reason_code=line.get("adjustment_reason")
        )
        payments.append(payment)

    # Calculate totals
```

```
    total_billed = sum(p.billed for p in payments)
    total_paid = sum(p.paid for p in payments)

    return ProcessedEOB(
        claim_id=claim_id,
        patient_id=patient_id,
        payments=payments,
        total_billed=total_billed,
        total_paid=total_paid,
        processing_date=datetime.utcnow()
    )
```

B Benchmark Suite Specification

We release our benchmark suite at `https://github.com/XY-Corp/CompiledAI`. The suite includes:

- **[N]** workflow specifications (YAML)
- Golden outputs for accuracy validation
- Measurement scripts for all seven metric categories
- Baseline implementations (Direct LLM, LangChain, Multi-agent)

C Healthcare Regulatory Context

As of July 2025, the FDA has authorized over 1,250 AI-enabled medical devices [FDA, 2025]. The Predetermined Change Control Plan (PCCP) guidance enables pre-specified algorithm updates without new submissions.

Our workflow automation systems generally fall outside FDA device jurisdiction (administrative processes, not clinical decisions), but code generation supports regulatory scrutiny: artifacts can be validated, documented, and audited as required by HIPAA and SOC 2.